



FORMULARIO DE PRODUCCIÓN DE CONTENIDOS

NOMBRE DE LA ASIGNATURA: Desarrollo en plataformas
NOMBRE COMPLETO DEL DOCENTE CONTENIDISTA: Damián Aníbal Nicolalde Rodríguez

DESARROLLO

1. INTRODUCCIÓN DE LA CLASE (2 párrafos)

El desarrollo web actual exige lenguajes capaces de responder de forma dinámica, modular y eficiente ante la complejidad de las aplicaciones modernas. En este contexto, JavaScript moderno (ES6+) representa una evolución significativa respecto a sus versiones anteriores, al incorporar nuevas formas de declarar variables, estructurar funciones y organizar el código en módulos reutilizables. Estas innovaciones no solo mejoran la legibilidad y seguridad del código, sino que además sientan las bases de la programación orientada a objetos y del trabajo asíncrono, permitiendo al desarrollador controlar mejor la lógica y el flujo de ejecución dentro del navegador.

Durante esta clase, el estudiante aprenderá a manejar los elementos centrales del lenguaje moderno: la correcta declaración de variables (`let` y `const`), la creación de funciones expresivas (tradicionales y flecha), el control de flujo mediante estructuras condicionales, y la ejecución de procesos asíncronos a través de Promesas y la sintaxis `async/await`. Se abordará también el manejo de errores como un componente esencial para la estabilidad de las aplicaciones, integrando el uso de bloques `try...catch`, clases personalizadas y buenas prácticas de depuración. Con ello, el estudiante comprenderá que dominar JavaScript moderno no se trata solo de escribir código que funcione, sino de escribir código confiable, mantenible y preparado para el cambio.

Clase 4: JavaScript moderno (ES6+)

RDA 1: Programar aplicaciones web dinámicas, interactivas y con diseño visualmente adaptable, que permita mejorar la experiencia del usuario.

Reto # 1

4.1. JavaScript moderno (ES6+)

JavaScript es el lenguaje que dota de interactividad a la web. Si HTML define la estructura y CSS el estilo, JavaScript (JS) es quien le da vida al navegador: valida formularios, reacciona a eventos, manipula el contenido, comunica con servidores y permite construir aplicaciones completas.

A lo largo del tiempo, JS ha evolucionado de un lenguaje básico de scripting a una



plataforma madura. Con la llegada de **ECMAScript 2015 (ES6)**, se introdujo un conjunto de mejoras que transformaron completamente su sintaxis, su capacidad de modularización y su potencia para la programación moderna.

El estándar **ECMAScript** es el conjunto de reglas que define cómo debe funcionar JavaScript. Cada versión (ES6, ES7, ES8, etc.) introduce nuevas funcionalidades que hacen el lenguaje más seguro, limpio y expresivo.

1. Evolución de JavaScript y el salto a ES6

Antes de ES6, escribir JavaScript implicaba una sintaxis más limitada. Por ejemplo, no existían clases, módulos, ni constantes, y las variables se declaraban solo con `var`, lo que generaba muchos errores por alcance (*scope*). ES6 incorporó nuevas estructuras que hoy son imprescindibles:

Tabla 1. Principales innovaciones introducidas en ES6

Categoría	Novedad	Ejemplo	Beneficio
Declaración de variables	<code>let, const</code>	<code>let nombre = "Ana"; const PI = 3.1416;</code>	Control de ámbito (<i>scope</i>) más seguro
Funciones modernas	Arrow functions	<code>const suma = (a,b) => a+b;</code>	Sintaxis más corta y clara
Plantillas de texto	Template literals	<code>`Hola, \${nombre}`</code>	Interpolación de variables
Desestructuración	Destructuring	<code>const {nombre, edad} = persona;</code>	Lectura directa de objetos o arreglos
Clases	<code>class</code>	<code>class Usuario { constructor(n){ this.nombre = n; }}</code>	Sintaxis orientada a objetos
Módulos	<code>export, import</code>	<code>import { suma } from "../util.js";</code>	Reutilización de código
Promesas	<code>Promise</code>	<code>fetch(url).then(...).catch(...);</code>	Manejo asíncrono más claro

Fuente: elaboración propia (2025).

2. Variables y ámbito (*scope*)

En ES6, el comportamiento de las variables cambió significativamente. Antes todo se declaraba con `var`, que tenía ámbito global o de función, provocando errores difíciles de depurar.

Con ES6, se introducen **let** y **const**:

```
// var: alcance global o de función
var x = 10;
if (true) {
  var x = 20;
}
console.log(x); // 20 → sobrescribe la variable global

// let: alcance de bloque
let y = 10;
if (true) {
  let y = 20;
  console.log(y); // 20 → variable local al bloque
}
console.log(y); // 10 → variable original se mantiene

// const: valor inmutable
const PI = 3.1416;
```

Resumen:

- Usa **let** para variables que cambian.
- Usa **const** para valores fijos.
- Evita **var** salvo en casos legacy.

3. Funciones modernas y Arrow Functions

Una de las mejoras más celebradas de ES6 son las funciones flecha (*arrow functions*). Permiten escribir funciones más concisas y legibles:

```
// Función tradicional
function saludar(nombre) {
  return "Hola " + nombre;
}

// Arrow function
const saludar = nombre => `Hola ${nombre}`;
```

Además, las arrow functions no alteran el contexto de **this**, lo cual evita muchos errores en programación orientada a objetos o con eventos DOM.

Ejemplo de uso de arrow function con contexto:

```
class Boton {
  constructor() {
    this.mensaje = "Click detectado";
  }
  activar() {
    document.querySelector("button")
      .addEventListener("click", () => {
        console.log(this.mensaje);
      });
  }
}
```



Aquí el `this` dentro de la función flecha apunta a la instancia de la clase, no al elemento DOM, garantizando un comportamiento consistente.

4. Template literals (plantillas literales)

Antes de ES6, concatenar cadenas era tedioso:

```
let nombre = "Ana";
console.log("Hola " + nombre + ", bienvenida al curso.");
```

Con ES6:

```
console.log(`Hola ${nombre}, bienvenida al curso.`);
```

Ventajas:

- Soporta multilínea.
- Permite interpolación de variables directamente.
- Mejora la legibilidad del código.

5. Desestructuración (Destructuring)

La desestructuración permite extraer valores de objetos o arreglos fácilmente:

```
const persona = { nombre: "Ana", edad: 28 };
const { nombre, edad } = persona;
console.log(nombre, edad); // Ana 28
```

También puede usarse con arreglos:

```
const numeros = [1, 2, 3];
const [a, b, c] = numeros;
```

Y combinarse con funciones:

```
function mostrar({ nombre, edad }) {
  console.log(`${nombre} tiene ${edad} años`);
}
```

6. Clases y Programación Orientada a Objetos

Antes de ES6, las clases se simulaban con funciones y prototipos.

Con la nueva sintaxis:

```
class Usuario {
  constructor(nombre) {
    this.nombre = nombre;
  }
  saludar() {
    return `Hola, soy ${this.nombre}`;
  }
}
```



```
}  
}  
  
const usuario = new Usuario("Damián");  
console.log(usuario.saludar());
```

Esto facilita:

- Herencia (`extends`)
- Métodos (`super()`)
- Organización modular del código

Ejemplo con herencia:

```
class Admin extends Usuario {  
  constructor(nombre, rol) {  
    super(nombre);  
    this.rol = rol;  
  }  
  mostrarRol() {  
    return `${this.nombre} es ${this.rol}`;  
  }  
}
```

7. Módulos (`import/export`)

Los módulos permiten dividir el código en archivos reutilizables:

archivo `util.js`

```
export const sumar = (a, b) => a + b;  
export const restar = (a, b) => a - b;
```

archivo `principal.js`

```
import { sumar } from './util.js';  
console.log(sumar(2, 3));
```

Esto mejora la organización, la reutilización y la mantenibilidad.

8. Diagramas de apoyo

```
graph TD  
A[Declaraciones modernas<br>let / const] --> B[Funciones modernas<br>Arrow Functions]  
B --> C[Clases y Objetos]  
C --> D[Módulos<br>import / export]  
D --> E[Programación estructurada<br>y mantenible]
```

Imagen 1. Flujo conceptual del uso de ES6

Fuente: Elaboración propia (2025)



9. Buenas prácticas al usar ES6

Categoría	Recomendación	Ejemplo
Nombres	Usa nombres descriptivos	<code>let temperaturaActual = 22</code>
Constantes	Usa mayúsculas para constantes	<code>const MAX_INTENTOS = 3;</code>
Código limpio	Evita repeticiones y anidaciones excesivas	Usa funciones puras y reutilizables
Seguridad	No uses <code>var</code>	Prefiere <code>let</code> y <code>const</code>
Claridad	Usa template literals	<code>`Hola \${usuario}`</code>
Estandarización	Aplica módulos y estructura clara	<code>import {función}</code>

10. Conclusión

ES6 no solo simplificó la sintaxis de JavaScript, sino que lo convirtió en un lenguaje moderno, modular y expresivo. Gracias a estas mejoras, los desarrolladores pueden escribir código más legible, seguro y mantenible, facilitando el trabajo colaborativo y la escalabilidad de los proyectos.

El dominio de estas bases es fundamental antes de avanzar a la Clase 5, donde se trabajará con el DOM y eventos, conectando la lógica del lenguaje con la estructura visual del documento.

4.2. Variables, funciones y estructuras de control

En JavaScript moderno (ES6+), comprender cómo se almacenan los datos, cómo se estructuran las instrucciones y cómo se controla el flujo del programa es la base de toda programación dinámica.

Las variables son los contenedores de información, las funciones permiten reutilizar código y las estructuras de control (condicionales y bucles) determinan la lógica de ejecución.

Esta clase profundiza en cómo el estándar ES6 redefinió la forma de declarar variables (`let`, `const`), estructurar funciones (clásicas, flecha y anónimas) y controlar la secuencia de ejecución mediante estructuras como `if`, `for`, `while` y `switch`. Estos fundamentos preparan el camino para manipular el DOM y manejar eventos en las próximas clases.

1. Variables: declaración, ámbito y mutabilidad

Una variable es un espacio de memoria donde se guarda un valor. En versiones anteriores de JavaScript se utilizaba únicamente `var`, lo cual causaba muchos problemas por su comportamiento de ámbito global o de función.

Con ES6, las palabras clave `let` y `const` solucionaron esos inconvenientes:



Tabla 1. Comparación de declaraciones de variables

Declaración	Reasignación	Re-declaración	Ámbito	Uso recomendado
var	Sí	Sí	Global o función	Evitar en ES6
let	Sí	No	Bloque { }	Variables cambiantes
const	No	No	Bloque { }	Valores fijos o referencias

Ejemplo práctico

```
var contador = 10;
if (true) {
  var contador = 20; // Sobrescribe la variable global
  console.log(contador); // 20
}
console.log(contador); // 20 (error lógico)
```

Con **let**, el comportamiento es diferente:

```
let contador = 10;
if (true) {
  let contador = 20; // Solo existe dentro del bloque
  console.log(contador); // 20
}
console.log(contador); // 10
```

Este ejemplo demuestra que **let** crea variables de bloque, manteniendo aisladas sus instancias dentro de cada contexto.

Y **const** protege los valores que no deben cambiar:

```
const PI = 3.1416;
PI = 3; // Error: no se puede reasignar un valor constante
```

```
graph TD
  A[Ámbito Global] --> B[Ámbito de Función]
  B --> C[Ámbito de Bloque]
  C --> D[Variables locales (let, const)]
```

Imagen 1. Jerarquía de ámbitos en JavaScript

Fuente: Elaboración propia (2025)

2. Tipos de datos

JavaScript tiene tipado dinámico, lo que significa que no se declara el tipo de variable:

```
let edad = 25; // número
edad = "veinticinco"; // cadena
```

Tabla 2. Tipos primitivos y complejos

Categoría	Tipo	Ejemplo	Descripción
-----------	------	---------	-------------



Primitivo	string	"Hola"	Texto o cadena
Primitivo	number	42	Enteros o decimales
Primitivo	boolean	true, false	Lógico
Primitivo	undefined	let x;	Variable declarada sin valor
Primitivo	null	let y = null;	Ausencia intencional de valor
Primitivo	symbol	Symbol("id")	Valor único (ES6)
Complejo	object	{nombre: "Ana"}	Colección de pares clave-valor
Complejo	array	[1, 2, 3]	Lista ordenada de elementos
Complejo	function	function() {}	Bloque reutilizable de código

3. Funciones: la base de la modularidad

Una función agrupa instrucciones que pueden ejecutarse varias veces en distintos contextos. Son fundamentales para reutilizar código, organizar lógica y reducir errores.

Sintaxis clásica

```
function saludar(nombre) {
  return `Hola, ${nombre}`;
}
console.log(saludar("Damián"));
```

Expresiones de función

```
const sumar = function(a, b) {
  return a + b;
};
```

Funciones flecha (arrow functions)

Introducidas en ES6, simplifican la sintaxis y heredan el contexto (this):

```
const sumar = (a, b) => a + b;
const cuadrado = x => x * x;
```

Ventajas de las arrow functions

- Sintaxis más corta y clara.
- Heredan el contexto de this.
- Ideales para funciones anónimas o callbacks.

```
graph TD
  A[Funciones tradicionales<br>function()] --> B[Funciones anónimas<br>const f = function(){}]
  B --> C[Arrow functions<br>const f = ()=>{}]
  C --> D[Funciones modernas con contexto<br>this heredado]
```

Imagen 2. Evolución de las funciones

Fuente: Elaboración propia (2025)



4. Parámetros y valores por defecto

ES6 permite asignar valores predeterminados a los parámetros:

```
function saludar(nombre = "invitado") {  
  console.log(`Bienvenido, ${nombre}`);  
}  
saludar(); // Bienvenido, invitado  
saludar("Ana"); // Bienvenido, Ana
```

Y con la desestructuración, se pueden recibir objetos de manera más legible:

```
function mostrar({nombre, edad}) {  
  console.log(`${nombre} tiene ${edad} años.`);  
}  
  
mostrar({nombre: "Carlos", edad: 30});
```

5. Estructuras de control

Las estructuras de control determinan qué instrucciones se ejecutan y cuántas veces, dependiendo de condiciones o bucles.

Condicionales

```
let edad = 18;  
if (edad >= 18) {  
  console.log("Mayor de edad");  
} else {  
  console.log("Menor de edad");  
}
```

También se puede usar `switch` para múltiples opciones:

```
let color = "rojo";  
switch (color) {  
  case "rojo": console.log("Detente"); break;  
  case "amarillo": console.log("Precaución"); break;  
  case "verde": console.log("Avanza"); break;  
  default: console.log("Color desconocido");  
}
```

Bucles

5.1. *for tradicional*

```
for (let i = 0; i < 3; i++) {  
  console.log(`Iteración ${i}`);  
}
```

5.2. *for...of*

Recorre arreglos o colecciones iterables:

```
let frutas = ["manzana", "pera", "uva"];  
for (let fruta of frutas) {  
  console.log(fruta);  
}
```



```
}
```

5.3. *for...in*

Recorre las propiedades de un objeto:

```
let persona = { nombre: "Ana", edad: 25 };
for (let clave in persona) {
  console.log(`${clave}: ${persona[clave]}`);
}
```

5.4. *while y do...while*

```
let contador = 0;
while (contador < 3) {
  console.log(contador);
  contador++;
}
let i = 0;
do {
  console.log(i);
  i++;
} while (i < 3);
```

6. Control de flujo con operadores lógicos y ternarios

```
let edad = 20;
let mensaje = (edad >= 18) ? "Acceso permitido" : "Acceso denegado";
console.log(mensaje);
```

Los operadores lógicos (&&, ||, !) también permiten tomar decisiones rápidas:

```
let usuario = "Damián";
usuario && console.log(`Hola, ${usuario}`);
```

7. Estructuras anidadas y buenas prácticas

- Evitar anidar muchos `if` dentro de `if`; usar `switch` o retornos anticipados.
- Limitar los bucles infinitos y usar condiciones claras.
- Combinar funciones y estructuras de control para mantener el código modular.

Ejemplo integrador

```
function verificarEdad(edad) {
  if (edad < 0) return "Edad no válida";
  if (edad < 18) return "Menor de edad";
  return "Mayor de edad";
}
```

8. Diagrama del flujo de control en JavaScript

```
graph TD
  A[Inicio] --> B[Declaración de variables]
  B --> C{Condición?}
  C -->|Verdadero| D[Ejecución del bloque de código]
  C -->|Falso| E[Salida o siguiente instrucción]
  D --> F[Reevaluar o continuar]
  E --> G[Fin del programa]
```

Imagen 3. Flujo de control en JavaScript

Fuente: Elaboración propia (2025)

9. Buenas prácticas

Categoría	Recomendación	Ejemplo
Variables	Usa <code>let</code> y <code>const</code>	<code>const nombre = "Ana";</code>
Funciones	Una función debe hacer una sola cosa	Evita funciones gigantes
Reutilización	Divide código en módulos	<code>import {funcion}</code>
Condicionales	Prefiere operadores ternarios simples	<code>(x>0) ? "Positivo" : "Negativo"</code>
Claridad	Indenta correctamente y usa nombres significativos	<code>edadUsuario, promedioNotas</code>

10. Conclusión

Las variables, funciones y estructuras de control constituyen el corazón de cualquier lenguaje de programación. En JavaScript moderno, la correcta elección entre `let` y `const`, el dominio de las funciones modernas (flecha, predeterminadas, desestructuradas) y el uso eficiente de estructuras condicionales y de repetición son los pilares para desarrollar código limpio y mantenible.

Estos conocimientos preparan al estudiante para las próximas etapas: el manejo del DOM, los eventos y la comunicación con el servidor (Fetch API), donde estas estructuras cobrarán vida al interactuar con elementos reales del documento.

4.3. Promesas, `async/await`

Introducción

En el desarrollo web moderno, muchos procesos no ocurren de inmediato. Cuando un navegador solicita datos a un servidor, lee un archivo, o espera la respuesta de una API, ese tiempo de espera no debe bloquear el resto del código. Para resolver esta necesidad, JavaScript utiliza un modelo de ejecución asíncrono, que permite que el programa continúe funcionando mientras se completan otras tareas en segundo plano.

Antes de ES6, este tipo de comportamiento se manejaba mediante callbacks (funciones pasadas como argumento a otras), pero con el tiempo se volvieron difíciles de mantener —lo que se conoce como el *callback hell*. ES6 introdujo una solución más elegante y legible: las Promesas, y posteriormente, `async/await`, que simplifican aún más la lectura y control de flujos asíncronos.

1. ¿Qué es la asincronía?

JavaScript utiliza un modelo *single-threaded* (un solo hilo de ejecución). Esto significa que solo puede ejecutar una tarea a la vez, pero gracias a su Event Loop (bucle de



eventos), puede programar tareas que se ejecutan “más adelante”, cuando otras hayan terminado.

Ejemplo clásico:

```
console.log("Inicio");
setTimeout(() => {
  console.log("Procesando...");
}, 2000);
console.log("Fin");
```

Salida:

```
Inicio
Fin
Procesando...
```

Aunque el `setTimeout` parece estar en medio del código, se ejecuta dos segundos después, sin bloquear la secuencia principal.

2. El problema del callback hell

Antes de ES6, el manejo asíncrono dependía de funciones anidadas:

```
obtenerDatosUsuario(function(usuario) {
  obtenerPedidos(usuario.id, function(pedidos) {
    procesarPedidos(pedidos, function(resultado) {
      console.log("Todo listo:", resultado);
    });
  });
});
```

Este enfoque funciona, pero es difícil de leer y mantener, sobre todo cuando hay múltiples niveles de anidación.

3. Promesas: el salto hacia la claridad

Una Promesa representa un valor que puede estar disponible ahora, más tarde, o nunca. Es un objeto que actúa como intermediario entre el código que solicita una tarea y el que entrega su resultado.

Sintaxis básica:

```
const promesa = new Promise((resolve, reject) => {
  const exito = true;
  if (exito) {
    resolve("Operación exitosa");
  } else {
    reject("Error en la operación");
  }
});

promesa
  .then(resultado => console.log(resultado))
```

```
.catch(error => console.log(error))
.finally(() => console.log("Proceso completado"));
```

```
stateDiagram-v2
    [*] --> pending
    pending --> fulfilled : resolve()
    pending --> rejected : reject()
    fulfilled --> [*]
    rejected --> [*]
```

Imagen 1. Flujo de estados de una Promesa

Fuente: Elaboración propia (2025)

Estados posibles de una Promesa

Estado	Descripción	Resultado
pending	La promesa está pendiente, aún no ha terminado	En espera
fulfilled	La promesa fue cumplida correctamente	resolve()
rejected	La promesa falló	reject()

Ejemplo práctico

```
function cargarDatos() {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            const conexionExitosa = true;
            if (conexionExitosa) {
                resolve("Datos cargados correctamente");
            } else {
                reject("Error al cargar datos");
            }
        }, 1500);
    });
}

cargarDatos()
    .then(res => console.log(res))
    .catch(err => console.log(err))
    .finally(() => console.log("Fin del proceso"));
```

Salida posible:

```
Datos cargados correctamente
Fin del proceso
```

4. Encadenamiento de Promesas

Las promesas pueden encadenarse para ejecutar operaciones secuenciales sin anidar callbacks.

```
function paso1() {
    return Promise.resolve("Paso 1 completado");
}
function paso2() {
    return Promise.resolve("Paso 2 completado");
}
function paso3() {
    return Promise.resolve("Paso 3 completado");
}
```



```
}

pasol()
  .then(res => { console.log(res); return paso2(); })
  .then(res => { console.log(res); return paso3(); })
  .then(res => console.log(res))
  .catch(err => console.error("Error:", err))
  .finally(() => console.log("Proceso terminado"));
```

Salida:

```
Paso 1 completado
Paso 2 completado
Paso 3 completado
Proceso terminado
```

5. Async/Await: simplificando la lectura

Aunque las promesas mejoraron la claridad, el método `then()` seguía fragmentando el código.

Con ES2017 (ES8) se incorporaron las palabras clave **async** y **await**, que permiten escribir código asíncrono como si fuera sincrónico, pero sin bloquear el hilo principal.

Regla principal:

- Toda función declarada con **async** devuelve una promesa.
- Dentro de una función **async**, **await** detiene la ejecución hasta que una promesa se resuelve.

Ejemplo comparativo:

```
// Con Promesas
cargarDatos()
  .then(res => console.log(res))
  .catch(err => console.log(err));

// Con async/await
async function ejecutar() {
  try {
    const resultado = await cargarDatos();
    console.log(resultado);
  } catch (error) {
    console.log(error);
  } finally {
    console.log("Proceso terminado");
  }
}

ejecutar();
```

Ventajas:

- Código más legible.
- Flujo lineal sin `then()` anidados.
- Mejor manejo de errores con `try...catch`.

graph TD

```
A[Inicio] --> B[Función async llamada]
B --> C[await promesa1]
C --> D[await promesa2]
D --> E[try/catch manejo de errores]
E --> F[Fin de la función]
```

Imagen 2. Flujo secuencial con async/await

Fuente: Elaboración propia (2025)

6. Ejemplo práctico integrador

Simulemos la carga de información de un usuario desde un servidor remoto:

```
function obtenerUsuario() {
  return new Promise((resolve) => {
    setTimeout(() => resolve({ id: 1, nombre: "Ana" }), 1000);
  });
}

function obtenerPedidos(idUsuario) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(["Pedido A", "Pedido B"]), 1500);
  });
}

async function mostrarInformacion() {
  try {
    console.log("Cargando información...");
    const usuario = await obtenerUsuario();
    const pedidos = await obtenerPedidos(usuario.id);
    console.log(`Usuario: ${usuario.nombre}`);
    console.log(`Pedidos: ${pedidos.join(", ")}`);
  } catch (error) {
    console.error("Error:", error);
  } finally {
    console.log("Fin del proceso");
  }
}

mostrarInformacion();
```

Salida:

```
Cargando información...
Usuario: Ana
Pedidos: Pedido A, Pedido B
Fin del proceso
```

Este ejemplo demuestra cómo async/await mantiene la secuencia lógica del código, aunque en realidad cada tarea se ejecuta de manera asíncrona y no bloqueante.

7. Errores comunes al usar Promesas y async/await

Error	Descripción	Solución
Olvidar await	La promesa no se resuelve antes de continuar	Anteponer await a cada promesa



No usar <code>try...catch</code>	Los errores no se manejan adecuadamente	Envolver el código con <code>try...catch</code>
No retornar en <code>then()</code>	Interrumpe el encadenamiento	Usar <code>return</code> para pasar resultados
Mezclar <code>then</code> y <code>await</code>	Confunde el flujo del programa	Elegir uno u otro, no ambos juntos

8. Tabla comparativa: Promesas vs async/await

Característica	Promesas (<code>then/catch</code>)	Async/Await
Sintaxis	Encadenada	Lineal
Legibilidad	Moderada	Alta
Mantenimiento	Requiere manejo cuidadoso de <code>then</code>	Más simple y estructurada
Manejo de errores	<code>catch()</code>	<code>try...catch</code>
Ideal para	Múltiples operaciones paralelas	Procesos secuenciales dependientes

9. Ejemplo combinado: Fetch API + async/await

En la práctica, `async/await` se usa principalmente con la Fetch API, para obtener datos desde un servidor:

```
async function cargarUsuarios() {
  try {
    const respuesta = await fetch("https://jsonplaceholder.typicode.com/users");
    const datos = await respuesta.json();
    console.table(datos.map(u => ({ id: u.id, nombre: u.name })));
  } catch (error) {
    console.error("Error al cargar usuarios:", error);
  }
}
cargarUsuarios();
```

Este código realiza una solicitud HTTP, espera la respuesta, la convierte a formato JSON y la muestra en forma tabular.

Es el mismo patrón que luego se aplicará en la Semana 5 cuando trabajemos con Fetch API y manipulación del DOM.

10. Buenas prácticas

- Usa `async/await` siempre que necesites un flujo secuencial.
- Muestra mensajes de carga o spinners mientras esperas datos.
- Envuelve todas las operaciones en `try...catch`.
- No mezcles `await` con `.then()`.
- No olvides que `await` solo funciona dentro de una función `async`.



```
A[Evento del usuario] --> B[Llamada asíncrona con fetch()]
B --> C[await respuesta]
C --> D[Procesamiento de datos JSON]
D --> E[Actualización del DOM]
E --> F[Fin del ciclo de interacción]
```

Imagen 3. Patrón de uso de Promesas y async/await

Fuente: Elaboración propia (2025)

11. Conclusión

El modelo asíncrono de JavaScript, basado en Promesas y mejorado con async/await, permite desarrollar aplicaciones más fluidas, rápidas y estables. Gracias a estas estructuras, el navegador no se bloquea mientras espera datos externos, garantizando una experiencia de usuario responsiva.

Dominar estas herramientas es esencial antes de avanzar hacia la Semana 5, donde el estudiante aplicará estos conceptos para interactuar dinámicamente con el DOM y consumir APIs reales.

4.4. Manejo de errores y buenas prácticas

En toda aplicación web, el error no es una posibilidad remota: es una certeza. Desde una variable mal escrita hasta una conexión interrumpida con un servidor, los errores pueden surgir en cualquier etapa del ciclo de vida de un programa. Por eso, un buen desarrollador no solo debe escribir código que funcione, sino también código que sepa fallar correctamente.

El manejo de errores en JavaScript busca identificar, capturar y responder a las situaciones inesperadas sin detener la ejecución completa del sistema. El objetivo es garantizar la resiliencia y estabilidad de la aplicación, brindando una buena experiencia al usuario y facilitando la depuración para el desarrollador.

1. Tipos de errores en JavaScript

JavaScript maneja principalmente tres categorías de errores:

Tipo de error	Descripción	Ejemplo	Solución
Errores de sintaxis	Fallos en la escritura del código	<code>if (x > 3 {</code>	Revisar paréntesis, comillas o llaves
Errores de referencia	Se accede a una variable o función no definida	<code>console.log(nombre); // nombre no definido</code>	Verificar ámbito o declaración previa



Errores de ejecución (runtime)	Ocurren durante la ejecución, aunque la sintaxis sea válida	<code>JSON.parse("{mal formado}");</code>	Capturar con <code>try...catch</code>
---------------------------------------	---	---	---------------------------------------

2. Manejo básico con `try...catch`

La estructura `try...catch` permite ejecutar código “protegido” y capturar cualquier error que ocurra dentro del bloque `try` sin detener el programa.

Sintaxis general

```
try {
  // Código que podría fallar
  let resultado = dividir(10, 0);
  console.log(resultado);
} catch (error) {
  // Manejo del error
  console.error("Ocurrió un error:", error.message);
}
```

Explicación

- El bloque `try` ejecuta el código normalmente.
- Si ocurre un error, se “lanza” automáticamente hacia el bloque `catch`.
- El objeto `error` contiene información como `name`, `message` y `stack`.

Ejemplo práctico

```
try {
  const json = '{ "nombre": "Ana", "edad": 25 }';
  const usuario = JSON.parse(json);
  console.log(usuario.nombre);
} catch (error) {
  console.error("Error al parsear JSON:", error.message);
}
```

Si el formato JSON estuviera mal escrito, el programa no se detendría, sino que mostraría un mensaje claro.

```
flowchart TD
  A[Inicio del programa] --> B[Bloque try]
  B -->|Sin error| C[Ejecución normal]
  B -->|Con error| D[Catch captura el error]
  D --> E[Mostrar mensaje o registrar en log]
  E --> F[Continuar ejecución]
  C --> F
```

Imagen 1. Flujo del manejo de errores

Fuente: Elaboración propia (2025)



3. Lanzar errores manualmente con `throw`

A veces no basta con esperar que ocurra un error; el desarrollador puede lanzar uno intencionalmente si detecta una condición inválida.

```
function dividir(a, b) {  
  if (b === 0) {  
    throw new Error("No se puede dividir por cero");  
  }  
  return a / b;  
}  
  
try {  
  console.log(dividir(10, 0));  
} catch (error) {  
  console.error("Error detectado:", error.message);  
}
```

Salida:

```
Error detectado: No se puede dividir por cero
```

Esto permite prevenir comportamientos peligrosos y crear validaciones personalizadas.

4. Bloque `finally`: limpieza y cierre seguro

El bloque `finally` se ejecuta siempre, ocurra o no un error. Es útil para liberar recursos, cerrar conexiones o limpiar variables.

```
try {  
  console.log("Abriendo conexión...");  
  throw new Error("Falla en la conexión");  
} catch (error) {  
  console.error("Error:", error.message);  
} finally {  
  console.log("Cerrando conexión...");  
}
```

Salida:

```
Abriendo conexión...  
Error: Falla en la conexión  
Cerrando conexión...
```

Esto garantiza que la aplicación mantenga integridad, incluso ante fallas.

5. Propiedades del objeto `Error`

Cada error tiene propiedades útiles para diagnóstico:

Propiedad	Descripción	Ejemplo
name	Tipo de error	"ReferenceError"
message	Descripción del error	"x is not defined"



stack	Rastreo del origen	Error at script.js:10:5
-------	--------------------	-------------------------

```
try {
  noExiste();
} catch (error) {
  console.log("Tipo:", error.name);
  console.log("Mensaje:", error.message);
  console.log("Rastro:", error.stack);
}
```

6. Errores personalizados

Puedes crear tipos de error propios para mejorar la semántica del programa:

```
class ErrorDeValidacion extends Error {
  constructor(mensaje) {
    super(mensaje);
    this.name = "ErrorDeValidacion";
  }
}

function validarEdad(edad) {
  if (edad < 0 || edad > 120) {
    throw new ErrorDeValidacion("Edad fuera de rango");
  }
  return "Edad válida";
}

try {
  console.log(validarEdad(200));
} catch (error) {
  console.error(`${error.name}: ${error.message}`);
}
```

Salida:

```
ErrorDeValidacion: Edad fuera de rango
```

Esto permite distinguir entre errores de conexión, validación, permisos, etc.

7. Manejo de errores asíncronos

En operaciones con Promesas o async/await, el manejo cambia ligeramente.

Con Promesas

```
fetch("https://api.ejemplo.com/datos")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(error => console.error("Error en la solicitud:", error));
```

Con async/await

```
async function cargarDatos() {
  try {
    const res = await fetch("https://api.ejemplo.com/datos");
    if (!res.ok) throw new Error("Respuesta no válida del servidor");
    const data = await res.json();
    console.log(data);
  } catch (error) {
    console.error("Error al cargar datos:", error.message);
  }
}
```



```
}
}
cargarDatos();
```

El uso de `try...catch` dentro de funciones `async` es la forma más clara y moderna de capturar errores asíncronos.

8. Registro y trazabilidad de errores

Una buena práctica profesional es registrar los errores (logging) para análisis posterior. Esto se puede hacer en la consola o enviando la información a un servidor de monitoreo.

```
function registrarError(error) {
  console.error(`[${new Date().toISOString()}] ${error.name}: ${error.message}`);
}

try {
  throw new Error("Fallo inesperado");
} catch (error) {
  registrarError(error);
}
```

Salida:

```
[2025-10-07T12:45:30.000Z] Error: Fallo inesperado
```

9. Buenas prácticas generales

Categoría	Recomendación	Ejemplo
Validaciones	Verifica entradas antes de procesar	<code>if (!email.includes("@")) throw new Error("Correo inválido");</code>
Modularidad	Divide código en funciones pequeñas	Evita bloques <code>try...catch</code> extensos
Transparencia	Muestra errores amigables al usuario	"No se pudo conectar al servidor"
Seguridad	No expongas mensajes internos en producción	Usa logs privados
Prevención	Utiliza <code>typeof</code> y <code>instanceof</code> para verificar tipos	<code>if (typeof x !== "number") throw ...</code>
Limpieza	Usa <code>finally</code> para cerrar conexiones o liberar memoria	<code>finally { cerrarConexion(); }</code>

```
flowchart TD
  A[Inicio del código] --> B[Validación de entradas]
  B --> C{Ocurre error?}
  C -->|Sí| D[Lanzar error con throw]
  C -->|No| G[Ejecutar flujo normal]
  D --> E[Captura con try...catch]
  E --> F[Registrar o mostrar mensaje]
  F --> H[Finalizar con finally]
  G --> H
```

Imagen 2. Ciclo completo del manejo de errores

Fuente: Elaboración propia (2025)

10. Depuración (debugging) y herramientas del navegador

Los navegadores modernos (como Chrome o Firefox) incluyen una pestaña “Sources” o “Depurador”, que permite:

- Colocar puntos de interrupción (breakpoints).
- Examinar el valor de variables en tiempo real.
- Seguir la pila de llamadas (call stack).
- Ver la traza exacta de un error.

Usar el depurador es más eficiente que saturar el código con `console.log()`.

```
function calcularPromedio(notas) {  
  debugger; // pausa la ejecución aquí  
  let suma = notas.reduce((a, b) => a + b, 0);  
  return suma / notas.length;  
}  
calcularPromedio([10, 9, 8]);
```

11. Patrón de buenas prácticas en producción

En aplicaciones reales se combinan todas estas estrategias:

```
async function procesarFormulario(datos) {  
  try {  
    validarCampos(datos);  
    const respuesta = await enviarDatos(datos);  
    mostrarMensaje("Formulario enviado con éxito");  
  } catch (error) {  
    registrarError(error);  
    mostrarMensaje("Ocurrió un problema. Inténtalo más tarde.");  
  } finally {  
    limpiarFormulario();  
  }  
}
```

Este patrón resume un flujo seguro, controlado y profesional:

1. Validar.
2. Ejecutar.
3. Capturar errores.
4. Informar al usuario.
5. Limpiar recursos.



12. Conclusión

El manejo de errores es una habilidad esencial para garantizar la estabilidad de las aplicaciones web. En JavaScript moderno, las estructuras `try...catch...finally`, combinadas con la posibilidad de lanzar errores personalizados y manejar fallos asíncronos mediante `async/await`, permiten construir sistemas robustos, seguros y mantenibles.

Además, aplicar buenas prácticas de validación, registro y depuración asegura que los errores no se conviertan en fallas críticas, sino en oportunidades para mejorar la calidad del código.

Referencias citadas en la Clase 4.

- Flanagan, D. (2020). *JavaScript: The Definitive Guide (7th ed.)*. O'Reilly Media.
- Mozilla Developer Network (MDN). (2025). *Guía de ECMAScript 2015 (ES6)*. Disponible en: <https://developer.mozilla.org/es/docs/Web/JavaScript>
- Crockford, D. (2019). *How JavaScript Works: The Mechanics of the World's Most Popular Programming Language*. Pelican Books.
- ECMAScript International. (2023). *ECMA-262, 14th Edition: ECMAScript Language Specification*. Ecma International.
- Freeman, E., & Robson, E. (2022). *Head First JavaScript Programming*. O'Reilly Media.

Definición de términos

- **Asincronía:** Capacidad de un lenguaje de programación para ejecutar tareas sin bloquear el flujo principal del programa. En JavaScript, la asincronía permite manejar operaciones como solicitudes HTTP, temporizadores o lectura de archivos, garantizando una experiencia fluida en la interfaz del usuario.
- **Promesa (Promise):** Objeto de JavaScript que representa la eventual finalización (o falla) de una operación asíncrona. Las promesas facilitan la ejecución de tareas dependientes mediante los métodos `.then()`, `.catch()` y `.finally()`, evitando la anidación excesiva de funciones (*callback hell*).

Enlaces externos recomendados

Título del enlace relacionado:

Guía oficial de ECMAScript 2015 (ES6) – Mozilla Developer Network (MDN)

Descripción del enlace relacionado:

Explicación completa y ejemplos detallados de las nuevas características introducidas en ES6: variables, funciones, clases, módulos y promesas.

Enlace:

<https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Classes>



Título del enlace relacionado:

JavaScript.info – Tutorial moderno de JavaScript

Descripción del enlace relacionado:

Curso gratuito y actualizado que explica los conceptos modernos del lenguaje JavaScript, incluyendo asincronía, manejo de errores y prácticas de desarrollo profesional.

Enlace:

<https://es.javascript.info>